CNN 실습 개요 소개

실습순서

1. 파이토치

1. 합성곱 신경망 CNN 입문 (Fashion-MNIST CNN 이미지분류 - 흑백)
2. 합성곱 신경망 CNN 확장 (CIFAR-10 CNN 이미지 분류 - 컬러)
3. 전이학습 커스텀 이미지 분류 (폴더구조 사용, 별도 라벨명 CSV 파일 사용)
4. 커스텀 이미지분류 모델 불러오기

2. Tensorflow

1. 합성곱 신경망 CNN 입문 (Fashion-MNIST CNN 이미지분류 - 흑백)
2. 전이학습 커스텀 이미지 분류
3. 커스텀 이미지분류 모델 불러오기

MLP(다층 퍼셉트론) vs CNN(합성곱 신경망)

이전 심층신경망을 이용한 회귀와 분류는 다층 퍼셉트론은 MLP 방식으로 DNN 모델을 구성하였음.
이미지 처리에 사용하는 심층망은 CNN 방식으로 구성하는 것이 표준임.

MLP(다층 퍼셉트론)와 CNN(합성곱 신경망)은 인공신경망의 종류이지만, 데이터 처리 방식과 구조에서 큰 차이가 있음.
MLP는 모든 뉴런이 서로 연결된 완전 연결 구조(Fully Connected)로 1차원 데이터를 처리하지만,
CNN은 필터를 공유하여 이미지의 2차원 공간 정보를 유지하며 국소적 특징을 추출함.

구조와 동작 방식의 차이

MLP : 입력 데이터를 1차원(Flatten)으로 쭉 펴서 사용.모든 층의 노드가 빽빽하게 연결(Fully Connected)되어 있음.데이터 전체의 전반적인 패턴을 학습함.
CNN : 이미지의 2차원(또는 3차원) 형태와 픽셀 간의 공간 정보를 유지.작은 크기의 필터(커널)를 이미지 전체에 윈도우처럼 훑어가며 특징을 추출. 같은 필터를 여러 위치에 재사용하는 '가중치 공유(Weight Sharing)'를 사용.

주요 특징 비교

파라미터 수: MLP는 연결이 많아 파라미터가 매우 많지만, CNN은 가중치 공유 덕분에 파라미터 수가 훨씬 적음.
공간 정보 보존: MLP는 이미지를 펴는 과정에서 위치/공간 정보가 망가지지만, CNN은 인접한 픽셀의 관계를 그대로 보존하며 학습함.
위치 불변성: CNN은 객체가 어디에 있든 필터가 찾아내므로 위치 변화에 강하지만, MLP는 위치가 바뀌면 처음부터 다시 배워야 할 정도로 취약함.

활용 분야

MLP: 정형 데이터(숫자 표, 가계부 데이터 등)나 단순한 분류 및 회귀 문제에 주로 사용.
CNN: 사진, 영상 같은 컴퓨터 비전(이미지 분류, 객체 검출 등) 분야의 표준 모델.

(참고로). 시계열 데이터는 RNN 방식(순환신경망)이 사용됨

PyTorch 01번 실습 - CNN Fashion-MNIST 패션 이미지 분류 (흑백)

딥러닝으로 [28×28 흑백 패션 이미지]의 [선, 모서리, 형태, 질감] 등의 특징을 추출하여 [10가지 의류 종류] 분류 모델 만들기

기존 DNN이 입력된 수치형 특성을 조합하여 학습했다면, CNN은 이미지에 합성곱 필터(Convolution Kernel)를 적용하여 시각적 특징을 자동으로 추출하고 분류
합성곱 신경망(Convolution Neural Network)을 이용한 이미지 특징 추출 및 다중분류(Image Classification)

[학습 데이터] torchvision Fashion-MNIST 데이터셋
MNIST 손글씨 숫자 데이터셋과 동일한 28×28 구조를 사용하는 패션 이미지 데이터셋
입력 데이터(X): 28×28 픽셀의 흑백 이미지 (1채널)
정답 데이터(y): 티셔츠, 바지, 스웨터, 원피스, 코트, 샌들, 셔츠, 운동화, 가방, 앵클부츠 - 총 10개 클래스(0~9)
학습 데이터 60,000개 / 테스트 데이터 10,000개
transforms.ToTensor()로 이미지를 Tensor로 변환(전처리)하고 **DataLoader**로 미니배치 단위 학습 [마지막 슬라이드에 추가 소개]

[입력층] 이미지 Tensor (1, 28, 28)
1채널 흑백 이미지의 28×28 픽셀값 입력 - 이미지의 공간적 구조를 유지한 상태로 특징 추출

[합성곱층 #1] Conv2d(1, 32, 3) + ReLU
3×3 크기의 필터 32개를 이용하여 이미지에서 32개의 특징 맵(Feature Map) 추출: 선, 모서리, 경계, 밝기 변화 등 기초적인 시각적 특징 학습
padding=1로 이미지 크기 유지 → (32, 28, 28)

[풀링층 #1] MaxPool2d(2, 2)
2×2 영역에서 가장 큰 값만 남겨 중요한 특징은 유지하면서 이미지 크기 축소
28×28 → 14×14 (32개의 특징 맵 유지)

PyTorch 01번 실습 - CNN Fashion-MNIST 패션 이미지 분류 (흑백)

[합성곱층 #2] Conv2d(32, 64, 3) + ReLU
32개의 특징 맵을 입력받아 64개의 복합 특징 맵 생성: 옷의 윤곽, 소매 형태, 신발의 곡선, 가방의 형태 등 더 복잡한 패턴 학습
padding=1로 이미지 크기 유지 → (64, 14, 14)

[풀링층 #2] MaxPool2d(2, 2)
특징 맵의 공간 크기를 다시 절반으로 축소하여 계산량 감소 및 핵심 특징 유지
14×14 → 7×7 (64개의 특징 맵 유지)

[Flatten] 다차원 특징 맵을 1차원으로 변환
64개의 7×7 특징 맵을 일렬로 펼쳐 3,136개의 숫자로 변환
64 × 7 × 7 = 3,136개 특성 → 완전연결층(Fully Connected Layer)에 전달

[완전연결층] Linear(3136, 128) + ReLU
CNN에서 추출한 3,136개의 특징을 조합하여 128개의 핵심 분류 특징 학습: 의류 종류별 형태와 패턴을 구별하기 위한 복합 특징 구성

[출력층] Linear(128, 10) - 다중분류
128개의 핵심 특성을 기반으로 Fashion-MNIST의 10개 클래스에 대한 로짓값(Logits) 출력
10개 점수 중 가장 높은 값을 가진 클래스를 최종 예측 결과로 선택 - argmax()
필요한 경우 Softmax 함수로 각 클래스의 분류 확률 계산

PyTorch 01번 실습 - CNN Fashion-MNIST 패션 이미지 분류 (흑백)

[모델 구성] PyTorch nn.Module + nn.Sequential
CNN 모델을 크게 2개의 영역으로 나누어 구성
1. feature_extractor 특징 추출기 : Conv2d → ReLU → MaxPool2d → Conv2d → ReLU → MaxPool2d
2. classifier 분류기 : Flatten 이후 Linear(3136, 128) → ReLU → Linear(128, 10)

__init__()에서 각 신경망 구조를 정의하고 **forward()에서 특징 추출** → Flatten → 클래스 분류 순서로 순전파 수행

[손실함수 및 반복 학습] CrossEntropyLoss + Optimizer
10개 클래스의 예측 로짓값과 실제 정답을 비교하여 다중분류 손실 계산
미니배치 단위로 순전파 → 손실 계산 → 역전파 → 가중치 갱신을 반복하여 이미지 분류 성능 개선

기존 DNN에서는 이미 정리된 수치형 feature들을 입력받아 여러 은닉층에서 조합했다면,
CNN에서는 이미지의 픽셀에 합성곱 필터를 적용하여 선, 모서리, 형태 등 공간적인 특징을 자동으로 추출
합성곱층(Conv2d)에서 특징 추출 → 풀링층(MaxPool2d)에서 특징 압축 → 완전연결층(Linear)에서 최종 이미지 분류라는 CNN의 핵심 구조 학습

PyTorch로 흑백 이미지 분류 CNN을 직접 구현하여 이미지 Tensor의 채널 개념, Feature Map, 필터(커널), 풀링, Flatten 및 다중분류의 전체 학습 흐름 이해

[이미지 전처리 와 미니배치 - 다음 슬라이드 추가 소개 **]**
transforms.Compose는 이미지의 전처리 과정을 자동화하고, DataLoader는 전처리된 이미지를 미니배치로 묶어 신경망에 효율적으로 공급
이미지 다운로드 → Tensor 변환 및 정규화 → 미니배치 구성 → CNN 학습이라는 이미지 딥러닝의 기본 데이터 처리 흐름 이해

PyTorch 01번 실습 - CNN Fashion-MNIST 패션 이미지 분류 (흑백)

[이미지 전처리] transforms.Compose + ToTensor

이미지 데이터를 CNN 모델이 학습할 수 있는 Tensor 형태로 변환하는 전처리 파이프라인 구성

Compose 안에 리스트[] 형태로 전달된 변환 함수들은 지정한 순서대로 앞의 출력값이 뒤의 입력값으로 전달되며 차례대로 실행

이미지 크기 조정, 크롭, 텐서 변환, 정규화 등 복잡한 전처리 과정을 코드 한 줄(transform(image))로 깔끔하게 처리할 수 있도록 도와줌

예시

```
transform = transforms.Compose([
    transforms.Resize((226, 226)),          # 1. 이미지 크기를 226x226으로 변경
    transforms.RandomCrop(224),             # 2. 임의로 224x224 크기로 잘라냄 (데이터 증강)
    transforms.ToTensor(),                  # 3. PIL/NumPy 이미지를 0~1 범위의 텐서로 변환
    transforms.Normalize((0.5,), (0.5,))    # 4. 0~1 범위를 -1~1 범위로 정규화
])
```

이제 이미지에 transform만 적용하면 위 4가지 과정이 순서대로 자동 실행됨.

```
output_tensor = transform(input_image)
```

[미니배치 데이터 관리] DataLoader

대량의 이미지 데이터를 지정한 크기의 미니배치(Mini-batch)로 나누어 CNN 모델에 공급하는 데이터 로더 구성

batch_size=64: 이미지 64장씩 묶어서 학습 - 60,000장을 총 938개 배치로 나누어 처리

shuffle=True: 학습할 때마다 이미지 순서를 무작위로 섞어 데이터 순서에 따른 학습 편향 감소

shuffle=False: 테스트 데이터는 순서를 유지하여 일관된 모델 평가 수행

num_workers: 여러 CPU 프로세스를 이용하여 이미지 로딩과 전처리를 병렬 처리 가능

개별 이미지(1, 28, 28)를 자동으로 묶어 (배치 크기, 채널, 높이, 너비) 형태의 4차원 Tensor 생성

예: 이미지 64장 → (64, 1, 28, 28) 형태로 CNN 모델에 입력

전체 이미지를 한꺼번에 연산하는 대신 미니배치 단위로 순전파 → 손실 계산 → 역전파 → 가중치 갱신을 반복하여 메모리 사용과 연산 효율 관리

PyTorch 02번 실습 - CNN CIFAR-10 이미지 분류 (컬러)

딥러닝으로 [32×32 RGB 컬러 이미지]의 [색상, 선, 모양, 질감, 객체 형태] 등의 특징을 추출하여 [10가지 사물과 동물] 분류 모델 만들기

기존 Fashion-MNIST가 단순한 흑백 이미지 분류였다면, CIFAR-10은 색상과 배경, 객체의 방향과 크기 등이 다양한 실제 컬러 이미지를 분류
CNN 합성곱층 확장 및 데이터 증강(Data Augmentation)을 통한 이미지 분류 성능 개선

[학습 데이터] torchvision CIFAR-10 데이터셋
컴퓨터 비전 분야에서 이미지 분류 모델의 성능을 평가하는 대표적인 벤치마크 데이터셋
입력 데이터(X): 32×32 픽셀의 RGB 컬러 이미지 (3채널)
정답 데이터(y): 비행기, 자동차, 새, 고양이, 사슴, 개, 개구리, 말, 배, 트럭 - 총 10개 클래스
학습 데이터 50,000장 / 테스트 데이터 10,000장
각 클래스별 6,000장으로 균형 있게 구성

[이미지 전처리] transforms.Compose + Normalize
컬러 이미지의 픽셀값을 CNN 모델이 학습하기 적합한 Tensor로 변환하고 정규화
transforms.ToTensor(): RGB 이미지의 픽셀값(0~255)을 0.0~1.0 범위의 실수형 Tensor로 변환
transforms.Normalize((0.5,)*3, (0.5,)*3): R, G, B 각 채널의 픽셀값을 -1~1 범위로 정규화
원본 이미지(H, W, C)를 PyTorch 입력 형태인 (3, 32, 32) Tensor로 변환

[미니배치 데이터 관리] DataLoader
batch_size=128로 이미지 128장씩 묶어 학습하고, shuffle=True로 매 Epoch마다 학습 데이터 순서 변경
num_workers=2로 데이터 로딩과 전처리를 병렬 처리하여 GPU에 데이터를 효율적으로 공급

[입력층] 이미지 Tensor (3, 32, 32)
RGB 3개 채널의 32×32 컬러 이미지 입력 - 색상 정보와 픽셀의 공간적 구조를 함께 학습

PyTorch 02번 실습 - CNN CIFAR-10 이미지 분류 (컬러)

[합성곱층 #1] Conv2d(3, 32, 3) + ReLU + MaxPool2d
RGB 3채널 이미지를 입력받아 3×3 필터 32개로 기초적인 시각적 특징 추출: 색상 변화, 선, 모서리, 경계, 단순한 질감 등
특징 맵 크기: (32, 32, 32) → 풀링 후 (32, 16, 16)

[합성곱층 #2] Conv2d(32, 64, 3) + ReLU + MaxPool2d
32개의 특징 맵을 조합하여 64개의 복합 특징 추출: 동물의 털 질감, 자동차 윤곽, 날개 형태, 물체의 곡선과 색상 패턴 등
특징 맵 크기: (64, 16, 16) → 풀링 후 (64, 8, 8)

[합성곱층 #3] Conv2d(64, 128, 3) + ReLU + MaxPool2d
64개의 특징을 다시 조합하여 128개의 고차원 특징 추출: 동물의 얼굴 형태, 자동차와 트럭의 구조 차이, 비행기와 배의 복합적인 형태 등
특징 맵 크기: (128, 8, 8) → 풀링 후 (128, 4, 4)

[Flatten] 다차원 특징 맵을 1차원으로 변환
128개의 4×4 특징 맵을 일렬로 펼쳐 2,048개의 숫자로 변환 . 128 × 4 × 4 = 2,048개 특성 → 완전연결층(Fully Connected Layer)에 전달

[완전연결층] Linear(2048, 256) + ReLU + Dropout(0.5)
CNN에서 추출한 2,048개의 특징을 조합하여 분류에 유용한 256개의 핵심 특징 학습
Dropout(0.5)를 적용하여 학습 중 일부 뉴런 출력을 무작위로 제외함으로써 특정 특징에 과도하게 의존하는 현상과 과적합(Overfitting) 완화

[출력층] Linear(256, 10) - 다중분류
256개의 핵심 특징을 기반으로 CIFAR-10의 10개 클래스에 대한 로짓값(Logits) 출력
10개 클래스의 점수 중 가장 높은 값을 가진 클래스를 최종 분류 결과로 선택 - argmax()
필요한 경우 Softmax 함수로 각 클래스의 분류 확률 계산

PyTorch 02번 실습 - CNN CIFAR-10 이미지 분류 (컬러)

[모델 구성] PyTorch nn.Module + nn.Sequential

CNN 모델을 크게 2개의 영역으로 구성

features: Conv2d → ReLU → MaxPool2d 과정을 3회 반복하여 이미지 특징 추출

classifier: Flatten → Linear(2048, 256) → ReLU → Dropout(0.5) → Linear(256, 10)

[손실함수 및 반복 학습] CrossEntropyLoss + Optimizer

10개 클래스의 예측 로짓값과 실제 정답을 비교하여 손실 계산

GPU에서 미니배치 단위로 순전파 → 손실 계산 → 역전파 → 가중치 갱신을 반복하여 컬러 이미지 분류 모델 학습

[추가 실습] Data Augmentation을 이용한 분류 성능 개선

Fashion-MNIST보다 CIFAR-10의 분류 정확도가 낮게 나타날 수 있음 - RGB 색상, 복잡한 배경, 다양한 촬영 각도와 객체 크기 등으로 학습 난이도 증가

[데이터 증강] transforms.RandomHorizontalFlip + RandomCrop

원본 이미지를 다양하게 변형하여 새로운 학습 이미지가 있는 것과 유사한 효과를 제공

RandomHorizontalFlip(): 이미지를 무작위로 좌우 반전

RandomCrop(32, padding=4): 이미지 주변에 여백을 추가한 뒤 32×32 영역을 무작위로 잘라 위치 변화 학습

원본 이미지 → 좌우 반전 / 위치 이동 / 무작위 Crop → 다양한 이미지 변형 패턴 학습

학습 데이터에만 데이터 증강을 적용하고 테스트 데이터에는 기본 전처리만 적용

[성능 비교] 데이터 증강 전후 모델 평가

동일한 CNN 구조에서 데이터 증강 적용 전후의 학습 정확도, 테스트 정확도, 손실값을 비교하여 일반화 성능 개선 여부 확인

PyTorch 02번 실습 - CNN CIFAR-10 이미지 분류 (컬러)

이전 Fashion-MNIST 실습에서는 28×28 흑백 이미지에 2개의 합성곱층을 사용했다면, 이번 CIFAR-10 실습에서는 32×32 RGB 컬러 이미지에 3개의 합성곱층을 사용하여 더 복잡한 시각적 특징을 학습

합성곱층을 확장하여 고차원 특징을 추출하고, Dropout과 Data Augmentation을 적용하여 과적합을 완화하고 새로운 이미지에 대한 분류 성능을 개선하는 방법 학습

단순한 CNN 직접 구현에서 출발하여 향후 LeNet, VGG, ResNet, EfficientNet 등 대표적인 CNN 아키텍처 및 전이학습(Transfer Learning)으로 확장하기 위한 기초 실습

수행과제

교재 : 다양한 프로젝트로 끝장내는 파이토치 딥러닝

- PART3. chapter01 ~ 02 책읽고 따라하기 [p.114~ p.156]

이미지분류 신경망의 대표적 모델인 LeNet, VGG, ResNet모델의 신경망 구조를 살펴보기

신경망을 구조를 외울 필요는 없고 이런 것들이 그동안 AI 연구자들에 의해 발전되어 왔구나 라는 것을 이해하는 정도면 됨. 논문같은 것을 보는게 이런 구조를 보는 것임

다음에 학습할 전이학습에서 이런 대표적인 모델의 특성을 고려하여 베이스모델을 선택하는 것임.
이제는 AI와 함께 하기에 외우는 것을 예전처럼 할 필요는 없지만 마치 기술적 이론을 학습하듯 살펴보면 됨.

PyTorch 03번 실습 - 전이학습_커스텀 이미지분류 1_폴더구조사용

ImageNet으로 사전학습된 [ResNet152 CNN 모델]을 활용하여 [120종 강아지 이미지]의 품종을 분류하는 커스텀 이미지 분류 모델 만들기

기존 CNN 실습에서는 합성곱층을 직접 설계하고 처음부터 학습했다면, 이번에는 대규모 이미지 데이터로 미리 학습된 CNN 모델의 특징 추출 능력을 재사용하여 새로운 이미지 분류 문제 해결 - 전이학습(Transfer Learning)을 이용한 커스텀 이미지 분류

[학습 데이터] 120 Dog Breed Image Classification

Kaggle에서 제공하는 120종 강아지 품종 이미지 데이터셋 사용

입력 데이터(X): 비글, 치와와, 골든 리트리버, 시베리안 허스키 등 다양한 강아지 이미지

정답 데이터(y): 강아지 품종 120개 클래스(0~119)

품종별 하위 폴더에 이미지가 저장된 폴더 기반 데이터셋

이미지의 크기, 배경, 촬영 각도, 털 색상 등이 다양하여 세밀한 품종별 특징 학습 필요

[데이터 로딩] torchvision.datasets.ImageFolder

강아지 품종별 하위 폴더명을 자동으로 클래스 라벨로 인식하여 이미지와 정답 데이터 구성

ImageFolder(root=...): 지정한 폴더 아래의 하위 폴더들을 클래스별로 인식

dataset.classes: 자동으로 생성된 120개 클래스 이름 확인

dataset.class_to_idx: 폴더명과 정수형 라벨(0~119)의 매핑 정보 확인

CSV 라벨 파일이나 별도의 Custom Dataset 클래스 없이 이미지 데이터 로딩

[이미지 전처리] transforms.Compose + Normalize

다양한 크기의 강아지 이미지를 사전학습 ResNet152 모델의 입력에 적합하도록 전처리

Resize / Crop: 이미지를 모델 입력 크기인 224×224로 조정

ToTensor: RGB 이미지의 픽셀값을 0~1 범위의 Tensor로 변환

Normalize: ImageNet 사전학습 모델에 사용된 RGB 채널별 평균과 표준편차로 정규화

DataLoader: 미니배치 단위로 이미지를 GPU에 공급하여 효율적으로 학습

PyTorch 03번 실습 - 전이학습_커스텀 이미지분류 1_폴더구조사용

[사전학습 모델] ResNet152 + ImageNet

대규모 이미지 데이터셋 ImageNet으로 미리 학습된 ResNet152 모델 불러오기

ImageNet: 이미지 분류 연구에 사용되는 대규모 이미지 데이터셋

ResNet152: 152개 층으로 구성된 깊은 CNN 아키텍처로 잔차 연결(Residual Connection)을 이용하여 심층신경망 학습

ImageNet-1K의 1,000개 클래스를 구분하도록 학습된 가중치 사용

[특징 추출기] 사전학습 CNN 구조 재사용

기존 ResNet152가 학습한 이미지의 시각적 특징 추출 능력을 새로운 데이터셋에 활용

초기 합성곱층: 선, 모서리, 색상, 질감 등 기초적인 시각적 특징

중간 합성곱층: 털의 질감, 귀의 윤곽, 눈과 코의 형태 등 복합적인 특징

깊은 합성곱층: 얼굴 구조, 몸의 형태 등 품종 구분에 유용한 고차원적 패턴

처음부터 수많은 필터의 가중치를 학습하는 대신 사전학습된 특징 추출기를 재사용하여 새로운 분류 문제에 활용

[출력층 교체] Fully Connected Layer (1000 → 120)

model.fc = nn.Linear(model.fc.in_features, 120)

기존 ResNet152의 마지막 출력층인 1,000개 클래스 분류기를 제거하고 120개 강아지 품종에 맞는 새로운 출력층으로 교체

기존 출력층: ImageNet 1,000개 클래스

새로운 출력층: 강아지 품종 120개 클래스

각 품종에 대한 로짓값(Logits) 120개 출력

[전이학습] Feature Extraction + Fine-Tuning

기존 CNN의 학습된 가중치를 활용하면서 새로운 강아지 품종 분류 문제에 맞게 모델 학습

Feature Extraction: 사전학습된 CNN의 가중치를 고정하고 새롭게 교체한 출력층만 학습

Fine-Tuning: 사전학습된 CNN의 일부 또는 전체 층을 추가로 학습하여 새로운 데이터에 적응

먼저 출력층만 학습하는 방식으로 전이학습을 이해하고, 필요하면 일부 합성곱층까지 미세조정하여 성능 개선

PyTorch 03번 실습 - 전이학습_커스텀 이미지분류 1_폴더구조사용

[GPU 가속] CUDA / Google Colab
ResNet152는 많은 가중치와 합성곱 연산을 포함하므로 GPU를 활용하여 이미지 특징 추출과 학습 수행
model.to(device)로 모델을 GPU 메모리로 이동시키고 학습 이미지와 정답 Tensor도 동일한 device에서 처리

[손실함수 및 학습] CrossEntropyLoss + Optimizer
120개 품종에 대한 예측 로짓값과 실제 품종 라벨을 비교하여 다중분류 손실 계산
미니배치 단위로 순전파 → 손실 계산 → 역전파 → 가중치 갱신을 반복하여 커스텀 분류기 학습

[모델 평가] Loss, Accuracy, Precision, Recall, F1 Score
학습된 모델을 검증 및 테스트 데이터로 평가하여 새로운 강아지 이미지에 대한 분류 성능 확인
Loss: 실제 품종과 예측값의 손실
Accuracy: 전체 이미지 중 올바르게 분류한 비율
Precision: 특정 품종으로 예측한 이미지 중 실제 해당 품종의 비율
Recall: 실제 특정 품종 이미지 중 정확하게 찾아낸 비율
F1 Score: Precision과 Recall의 조화평균
120개 클래스별 평가 및 Macro Average F1 Score 등을 이용하여 품종별 분류 성능 비교

[학습 모델 저장] state_dict + .pth
전이학습으로 학습한 ResNet152 모델의 가중치와 버퍼를 .pth 파일로 저장
state_dict(): 모델의 학습된 가중치와 필요한 상태값을 딕셔너리 형태로 관리
torch.save(): 학습 결과를 파일로 저장
load_state_dict(): 동일한 모델 구조를 생성한 후 저장된 상태값을 다시 불러오기
model.eval(): 학습된 모델을 추론(Inference) 모드로 전환
향후 저장된 모델을 불러와 새로운 강아지 사진의 품종을 예측하는 프로그램으로 확장

PyTorch 03번 실습 - 전이학습_커스텀 이미지분류 1_폴더구조사용

기존 CNN 이미지 분류 실습에서는 합성곱층과 완전연결층을 직접 설계하고 모든 가중치를 처음부터 학습했다면,
전이학습에서는 대규모 데이터로 이미 학습된 CNN의 특징 추출 능력을 재사용하고 마지막 분류층을 새로운 문제에 맞게 변경
또한 이미지 파일을 품종별 폴더에 저장하고 ImageFolder로 자동 라벨링함으로써 별도의 CSV 파일이나 커스텀 데이터 로더 없이 자신만의 이미지 분류 데이터셋 구성

사전학습 모델 불러오기 → 이미지 전처리 → 출력층 교체 → 전이학습 → 모델 성능 평가 → 학습된 가중치 저장까지 실무에서 활용하는 커스텀 이미지 분류 모델 개발의 전체 흐름 이해

[전이학습 미세조정 전략 : 추가실습]

전이학습 미세조정 전략

[미세조정 전략] 데이터 규모와 이미지 유사성에 따른 학습 범위 결정
사전학습된 CNN 모델을 새로운 데이터셋에 적용할 때는 **새로운 데이터셋의 크기**와 **기존 학습 데이터와의 유사성**에 따라 학습시킬 레이어의 범위를 결정

Feature Extraction: 사전학습된 CNN 특징 추출기의 가중치를 고정(Freeze)하고 마지막 분류층(FC)만 학습
Fine-Tuning: 사전학습된 CNN의 일부 또는 전체 레이어의 동결을 해제(Unfreeze)하여 새로운 데이터셋에 맞게 가중치를 추가 학습

[데이터 유사성] 기존 모델이 학습한 특징의 재사용 가능성
유사성 높음: ImageNet의 동물 사진 → 강아지 120종 품종 분류 (기존 특징 재사용에 유리)
유사성 낮음: ImageNet의 자연 이미지 → 의료 MRI, 산업용 X-ray 등 (새로운 도메인의 특징 학습 필요)

[데이터 규모와 유사성에 따른 전이학습 전략]
[데이터가 적은 경우] 과적합 방지 중심
학습 데이터가 적고 기존 데이터와 유사하다면 특징 추출기를 고정하고 출력층만 학습하는 방식이 효율적
기존 데이터와 차이가 크다면 마지막 합성곱 블록 등 일부 레이어의 동결을 해제하여 새로운 특징을 학습하되, 과적합에 주의
[데이터가 많은 경우] 새로운 특징 학습 범위 확장
충분한 학습 데이터가 있다면 일부 또는 전체 CNN 레이어까지 Fine-Tuning하여 새로운 데이터셋에 적합한 특징 추출 능력 개선

데이터 규모	유사성 높음	유사성 낮음
적음	FC만 학습부터 시작	FC + 일부 상위 Layer 학습 검토
많음	FC + 일부 Layer Fine-Tuning 비교	전체 Fine-Tuning까지 단계적 비교

전이학습 미세조정 전략 - Fine Tuning

사전학습된 CNN의 기존 특징을 최대한 재사용하면서, 새로운 데이터셋의 특성에 따라 학습이 필요한 레이어만 선택적으로 조정하는 것이 전이학습의 핵심
데이터가 적을수록 과적합 위험을 고려하고, 데이터가 충분할수록 학습 범위를 확장하여 Feature Extraction과 Fine-Tuning의 성능을 비교

추가 실습 - ResNet152 일부 Layer Fine-Tuning

[Fine-Tuning] ResNet152의 마지막 합성곱 블록까지 학습하여 강아지 120종 분류 성능 개선하기

이전 실습에서는 사전학습된 ResNet152의 특징 추출기를 고정하고 마지막 FC 출력층만 학습했다면, 이번에는 마지막 합성곱 블록(Layer4)의 동결을 해제하여 새로운 이미지 특징까지 추가 학습

[모델 구조] ResNet152 학습 범위 변경
Conv1 → 가중치 고정(Freeze)
Layer1 → 가중치 고정(Freeze)
Layer2 → 가중치 고정(Freeze)
Layer3 → 가중치 고정(Freeze)
Layer4 → 동결 해제(Unfreeze), 추가 학습
FC → 120개 품종 분류, 학습

[일부 Layer 학습 허용] requires_grad = True
model.layer4.parameters()의 requires_grad를 True로 변경하여 역전파 과정에서 마지막 합성곱 블록의 가중치도 갱신하도록 설정
기존의 기초적인 시각적 특징은 유지하면서 강아지 품종 구분에 유용한 고차원 특징을 추가 학습

PyTorch 04번 실습 - 전이학습_커스텀 이미지분류 2_별도 라벨명 csv 파일 사용

[이미지 파일 + CSV 라벨 정보]를 이용하여 [120종 강아지 이미지]의 품종을 분류하는 커스텀 이미지 분류 모델 만들기

이전 실습에서는 품종별 하위 폴더명을 자동으로 라벨로 사용하는 ImageFolder 방식을 활용했다면, 이번에는 이미지 파일명(ID)과 정답 품종명이 별도의 CSV 파일에 저장된 데이터셋을 직접 처리하여 전이학습 수행 - PyTorch Custom Dataset 구현 및 ResNet152 전이학습

[학습 데이터] Kaggle Dog Breed Identification 데이터셋

Kaggle 이미지 분류 경진대회에서 제공하는 120종 강아지 품종 이미지 데이터셋 사용

입력 데이터(X): 다양한 강아지 품종의 RGB 컬러 이미지
정답 데이터(y): 120개 강아지 품종(0~119)
train 폴더: 라벨이 존재하는 이미지 10,222장
test 폴더: 정답 라벨이 공개되지 않은 이미지 10,357장
labels.csv: 이미지 파일명(ID)과 강아지 품종명(breed)의 매핑 정보
sample_submission.csv: Kaggle 경진대회에 제출할 120개 품종별 예측 확률 양식

Kaggle의 test 폴더에는 정답 라벨이 없으므로, 이번 실습에서는 라벨이 있는 10,222장의 이미지를 Train / Validation / Test로 분할하여 학습과 성능 평가 수행

[데이터 라벨 처리] CSV 파일 + pandas DataFrame

labels.csv에서 이미지 ID와 품종명을 읽어 각 이미지의 정답 라벨 구성

- id: 이미지 파일명과 연결되는 고유 식별자
- breed: 강아지 품종을 나타내는 문자열 라벨
- class_names: 120개 품종 이름을 정렬한 클래스 목록
- class_to_idx: 품종명을 0~119 사이의 숫자 라벨로 변환하는 매핑 정보

이미지 파일명(ID)으로 실제 이미지 경로를 찾고, CSV의 품종명을 숫자 라벨로 변환하여 신경망 학습에 활용

PyTorch 04번 실습 - 전이학습_커스텀 이미지분류 2_별도 라벨명 csv 파일 사용

[커스텀 데이터셋] torch.utils.data.Dataset 상속
PyTorch의 Dataset 클래스를 상속받아 이미지 파일과 CSV 라벨을 직접 연결하는 DogDataset 클래스 구현
__init__(): CSV 데이터프레임과 이미지 전처리 함수를 저장하여 데이터셋 초기화
__len__(): 데이터셋에 포함된 이미지 개수 반환
__getitem__(idx): 지정된 인덱스의 이미지 파일을 읽고 전처리한 후 숫자 라벨과 함께 반환
Image.open().convert("RGB")로 이미지 로딩 → transform()으로 전처리 → class_to_idx로 정답 라벨 변환
최종적으로 (이미지 Tensor, 정답 클래스 번호) 형태의 데이터를 반환하여 DataLoader에 공급

[이미지 전처리] ResNet152 Pretrained Transform
다양한 크기의 강아지 이미지를 ImageNet으로 사전학습된 ResNet152 모델의 입력 형식에 맞게 변환

[사전학습 모델] ResNet152 + ImageNet
ImageNet-1K 데이터셋으로 사전학습된 ResNet152 모델을 불러와 기존 CNN의 이미지 특징 추출 능력 재사용

[특징 추출기] ResNet152 Convolution Layers
사전학습된 합성곱층에서 이미지의 시각적인 특징 자동 추출. 최종적으로 이미지에서 2,048개의 특징을 가진 벡터 생성

[출력층 교체] Fully Connected Layer (2048, 120)
model.fc = nn.Linear(model.fc.in_features, 120)
기존 ImageNet 1,000개 클래스 분류층을 제거하고 강아지 120개 품종에 맞는 새로운 분류층으로 교체
2,048개의 이미지 특징을 조합하여 120개 품종에 대한 로짓값(Logits) 출력

PyTorch 04번 실습 - 전이학습_커스텀 이미지분류 2_별도 라벨명 csv 파일 사용

[전이학습] Feature Extraction + Fine-Tuning
기존 ResNet152의 특징 추출 능력을 재사용하면서 새로운 강아지 품종 분류 문제에 맞게 학습
먼저 기존 합성곱층을 동결(Freeze)하고 마지막 FC 분류층을 학습한 뒤, 필요하면 Layer4 등의 일부 합성곱층을 미세조정(Fine-Tuning)하여 분류 성능 개선

[손실함수 및 반복 학습] CrossEntropyLoss + Optimizer
120개 클래스의 예측 로짓값과 CSV에서 읽어온 실제 품종 라벨을 비교하여 다중분류 손실 계산
GPU에서 미니배치 단위로 순전파 → 손실 계산 → 역전파 → 가중치 갱신을 반복하여 이미지 분류 모델 학습

[모델 평가] Accuracy, Cross Entropy, Top-5 Accuracy, Macro F1, Confusion Matrix
라벨이 있는 데이터에서 분리한 테스트 이미지로 학습된 모델의 분류 성능 평가
Accuracy: 120개 품종 중 정답을 정확하게 예측한 비율
Cross Entropy: 실제 정답과 예측 결과의 손실값
Top-5 Accuracy: 예측 확률이 높은 상위 5개 품종 안에 실제 정답이 포함된 비율
Macro F1: 각 품종별 F1 Score를 계산한 후 동일한 비중으로 평균
Confusion Matrix: 실제 품종과 예측 품종을 비교하여 어떤 품종끼리 자주 혼동하는지 분석

이전 전이학습에서는 이미지를 품종별 하위 폴더에 저장하고 ImageFolder로 자동 라벨링했다면,
이번 실습에서는 이미지 파일명과 품종 정보가 저장된 CSV 파일을 연결하여 직접 Custom Dataset 클래스 구현
Kaggle 경진대회처럼 이미지와 정답 데이터가 별도로 제공되는 환경에서
이미지 로딩 → CSV 라벨 매핑 → Dataset 구현 → DataLoader 구성 → 전이학습 → 모델 평가의 전체 데이터 처리 및 학습 과정 이해

ImageFolder와 Custom Dataset의 차이를 비교하고, CSV 기반 이미지 데이터셋을 PyTorch 모델에 직접 연결하여 다양한 실무 이미지 분류 프로젝트에 적용하는 방법 학습

PyTorch 05번 실습 - 전이학습_커스텀 이미지분류 모델 불러오기

전이학습으로 학습하고 저장한 [전이학습 - 강아지 품종 분류 모델]을 불러와 [새로운 강아지 이미지]의 품종을 예측하는 이미지 분류 프로그램 만들기

이전 실습에서는 120종 강아지 이미지를 이용하여 ResNet152 모델을 전이학습하고 가중치를 저장했다면, 이번에는 저장된 모델의 가중치를 복원하여 추가 학습 없이 새로운 이미지의 품종을 예측하는 추론(Inference) 과정 구현

[학습 모델 파일] PyTorch state_dict + .pth
이전 실습에서 torch.save()로 저장한 강아지 품종 분류 모델의 가중치 파일 사용
dog_breed_model.pth: 전이학습으로 학습된 모델의 가중치와 상태 정보
state_dict(): 모델의 가중치, 편향 등 학습된 상태값을 딕셔너리 형태로 관리
load_state_dict(): 저장된 가중치를 동일한 신경망 구조에 적용하여 학습 결과 복원
class_names: 예측한 클래스 번호(0~119)를 실제 강아지 품종명으로 변환하는 목록

[모델 구조 복원] ResNet152 + Linear(2048, 120)
ImageNet 사전학습에 사용했던 ResNet152 구조를 다시 생성하고 마지막 출력층을 120개 품종에 맞게 변경
model.fc = nn.Linear(model.fc.in_features, 120)
기존 학습 모델과 동일한 구조를 만든 후 저장된 가중치를 불러와 학습된 120종 강아지 분류 모델 복원

[가중치 불러오기] model.load_state_dict()
torch.load()로 저장된 .pth 파일의 가중치를 읽고 model.load_state_dict()로 모델에 적용
학습을 처음부터 다시 수행하지 않고 이전에 학습된 CNN 특징 추출기와 최종 분류층의 가중치 및 상태 복원

[추론 모드] model.eval() + torch.no_grad()
model.eval()로 신경망을 평가 모드로 전환하여 Dropout과 BatchNorm 등이 추론에 맞게 동작하도록 설정

PyTorch 05번 실습 - 전이학습_커스텀 이미지분류 모델 불러오기

[새로운 이미지 입력] 강아지 사진 불러오기

학습에 사용하지 않은 새로운 강아지 사진(JPG, PNG 등)을 입력받아 품종 예측

Image.open(): 새로운 이미지 파일 불러오기
convert("RGB"): 이미지를 RGB 3채널로 변환
transform(): ResNet152 학습 시 사용한 입력 형식에 맞게 이미지 전처리
unsqueeze(0): 이미지 1장을 배치 형태인 (1, 3, 224, 224) Tensor로 변환
to(device): 이미지 Tensor를 모델과 동일한 CPU 또는 GPU 장치로 이동

[최종 예측] argmax() + class_names

120개 품종의 예측 확률 중 가장 높은 값을 가진 클래스 번호를 선택하고 실제 품종명으로 변환

torch.softmax(logits, dim=1): 120개 품종의 예측 확률 계산
torch.argmax(): 가장 높은 예측 확률을 가진 품종 번호 선택
class_names[predicted_idx]: 품종 번호를 실제 강아지 품종명으로 변환
예: Beagle 0.87 / Basset Hound 0.08 / 기타 품종 0.05 → Beagle(비글) 예측

이전 실습에서는 이미지 데이터셋 준비 → 전이학습 → 손실 계산 → 역전파 → 가중치 갱신 → 모델 저장까지 학습 과정을 수행했다면,
이번 실습에서는 모델 구조 복원 → 저장된 가중치 로딩 → 새로운 이미지 전처리 → 순전파 → Softmax 확률 계산 → 품종 예측이라는 추론 과정 구현
학습(Training)은 모델의 가중치를 최적화하는 과정이고, 추론(Inference)은 학습된 가중치를 이용하여 새로운 데이터의 결과를 예측하는 과정

학습된 모델을 저장하고 다시 불러와 실제 이미지 분류에 활용함으로써, 딥러닝 모델을 재사용하고 향후 웹 서비스나 이미지 분류 애플리케이션으로 확장하는 기본적인 모델 활용 흐름 이해

PyTorch 05번 실습 - 전이학습_커스텀 이미지분류 모델 불러오기

수행과제

교재 : 다양한 프로젝트로 끝장내는 파이토치 딥러닝

- PART3. chapter 03 ~ 05 읽고 따라하기. 여러사례 연습 가능 (p.157 ~ p.217)
- 원하는 이미지분류 용 데이터셋을 검색하여 다운로드 한 후 전이학습 시도해보기 .. *데이터셋크기와 유사도에 따라 미세조정 방식 결정해보기*
- 사전학습 모델 검색해보기

NEXT...

Tensorflow 모듈 CNN 및 전이학습.

Tensorflow 01번 실습 - CNN Fashion-MNIST 패션 이미지 분류 (흑백)

TensorFlow 로 [28×28 흑백 패션 이미지]의 특징을 추출하여 [10가지 의류 종류]를 분류하는 CNN 모델 만들기

이전 PyTorch CNN 실습에서 사용했던 동일한 Fashion-MNIST 데이터셋과 신경망 구조를 TensorFlow Keras로 구현하여 CNN 모델 구성 및 학습 방식 비교

[학습 데이터] Fashion-MNIST 데이터셋

입력 데이터(X): 28×28 픽셀의 흑백 이미지 (1채널), 정답 데이터(y): 티셔츠, 바지, 스웨터, 원피스, 코트, 샌들, 셔츠, 운동화, 가방, 앵클부츠 - 10개 클래스
학습 데이터 60,000장 / 테스트 데이터 10,000장, 픽셀값을 0~1로 변환하고 입력 이미지를 (28, 28, 1) 형태로 구성

[입력층] Input(shape=(28, 28, 1))

28×28 크기의 흑백 이미지 입력 - TensorFlow는 이미지 Tensor를 기본적으로 (높이, 너비, 채널) 순서로 구성
PyTorch는 (채널, 높이, 너비) 순서를 사용하므로 입력 데이터의 차원 순서가 다름

[합성곱층 #1] Conv2D(32, (3,3)) + ReLU + MaxPooling2D

3×3 필터 32개로 이미지의 선, 모서리, 경계 등 기초적인 시각적 특징 추출. 특징 맵 크기: (28, 28, 32) → 풀링 후 (14, 14, 32)

[합성곱층 #2] Conv2D(64, (3,3)) + ReLU + MaxPooling2D

32개의 특징 맵을 조합하여 옷의 윤곽, 소매, 신발과 가방의 형태 등 더 복잡한 특징 64개 추출. 특징 맵 크기: (14, 14, 64) → 풀링 후 (7, 7, 64)

[Flatten] 다차원 특징 맵을 1차원으로 변환

7 × 7 × 64 = 3,136개의 특징값으로 변환하여 완전연결층에 전달

[완전연결층] Dense(128) + ReLU

3,136개의 특징을 조합하여 의류 분류에 유용한 128개의 핵심 특성 학습

Tensorflow 01번 실습 - CNN Fashion-MNIST 패션 이미지 분류 (흑백)

[출력층] Dense(10) + Softmax
128개의 핵심 특성을 기반으로 10개 의류 클래스의 로짓값 계산 후 Softmax 함수로 각 클래스의 확률 출력
10개 확률값의 합은 1(100%)이며, 가장 높은 확률의 클래스를 최종 예측 결과로 선택 - argmax()

[모델 구성] TensorFlow Keras Sequential
Input(28,28,1) → Conv2D(32,3,ReLU) → MaxPooling2D(2) → Conv2D(64,3,ReLU) → MaxPooling2D(2) → Flatten() → Dense(128,ReLU) → Dense(10,Softmax)

PyTorch의 nn.Conv2d, nn.MaxPool2d, nn.Linear에 대응하는 TensorFlow의 Conv2D, MaxPooling2D, Dense를 이용하여 동일한 구조의 CNN 구현

[손실함수 및 학습] SparseCategoricalCrossentropy + Optimizer
model.compile()로 손실함수, 최적화 알고리즘 및 정확도(Accuracy) 평가 지표 설정
model.fit()으로 순전파 → 손실 계산 → 역전파 → 가중치 갱신 과정을 자동 수행
출력층에 Softmax를 포함하므로 SparseCategoricalCrossentropy(from_logits=False) 사용

[PyTorch와 TensorFlow CNN 구현 방식 비교]
모델 구성: PyTorch는 nn.Module과 forward()로 순전파를 정의하지만, TensorFlow Keras는 Sequential()에 레이어를 순서대로 추가하여 구성
이미지 차원: PyTorch는 (N, C, H, W), TensorFlow는 기본적으로 (N, H, W, C) 형태 사용
출력층: PyTorch는 일반적으로 로짓값을 그대로 출력하고, TensorFlow 실습에서는 Softmax로 확률값까지 계산
모델 학습: PyTorch는 학습 반복문을 직접 구현하지만, TensorFlow Keras는 compile()과 fit()으로 간결하게 구현

동일한 Fashion-MNIST 데이터와 CNN 아키텍처를 두 프레임워크로 구현하여 합성곱 신경망의 공통된 학습 원리를 확인하고, 모델 구성과 학습 API의 차이 이해

Tensorflow 02번 실습 - 전이학습을 이용한 한국음식 이미지 분류_폴더구조사용

ImageNet으로 사전학습된 [EfficientNetV2B0 CNN 모델]을 활용하여 [21종 한국음식 사진]의 종류를 분류하는 커스텀 이미지 분류 모델 만들기

기존 Fashion-MNIST 실습에서는 CNN의 합성곱층을 직접 설계하고 처음부터 학습했다면, 이번에는 사전학습된 EfficientNetV2B0 모델의 이미지 특징 추출 능력을 재사용하고 새로운 분류층을 추가하여 한국음식 이미지 분류 모델 구현 - 전이학습(Transfer Learning)

[학습 데이터] korean_food_photos 한국음식 이미지 데이터셋

직접 준비한 한국음식 사진을 이용하여 21개 음식 종류를 분류하는 지도학습 수행

입력 데이터(X): 다양한 한국음식의 RGB 컬러 이미지

정답 데이터(y): 비빔밥(bibimbap), 불고기(bulgogi), 김치(kimchi), 삼겹살(samgupsal), 김밥(gimbob) 등 총 21개 클래스

음식 종류별 하위 폴더에 이미지를 저장하고 폴더명을 정답 라벨로 사용

[데이터 로딩] image_dataset_from_directory()

tf.keras.utils.image_dataset_from_directory()를 이용하여 음식 종류별 하위 폴더명을 자동으로 클래스 라벨로 변환

labels='inferred': 하위 폴더명을 자동으로 정답 라벨로 인식

image_size=(224,224): 모델 입력에 맞게 이미지 크기 변경

class_names: 자동 생성된 21개 음식 클래스 이름 확인

validation_split: 학습 데이터와 검증 데이터 분할 가능

PyTorch의 ImageFolder와 유사하게 별도의 CSV 라벨 파일 없이 폴더 구조만으로 이미지 분류 데이터셋 구성

[사전학습 모델] EfficientNetV2B0 + ImageNet

input_shape=(224,224,3): RGB 3채널 컬러 이미지 입력

include_top=False: 기존 1,000개 클래스 분류층을 제외하고 CNN 특징 추출기만 사용

weights='imagenet': 사전학습된 합성곱 신경망의 가중치 재사용

Tensorflow 02번 실습 - 전이학습을 이용한 한국음식 이미지 분류_폴더구조사용

[입력층] InputLayer(shape=(224,224,3))
224×224 크기의 RGB 한국음식 사진을 입력받아 이미지 특징 추출기로 전달

[특징 추출기] EfficientNetV2B0
사전학습된 CNN의 여러 합성곱층을 통해 음식의 색상, 질감, 재료 형태, 조리 상태, 음식의 전체적인 모양 등 복합적인 시각적 특징 추출
기존 ImageNet에서 학습한 특징을 재사용하여 한국음식 이미지의 분류에 활용

[추가 합성곱층 #1] Conv2D(128, 1×1) + ReLU
사전학습 모델에서 추출한 특징 맵을 1×1 합성곱 필터 128개로 조합하고 채널 수를 128개로 변환
이미 학습된 고차원 특징을 한국음식 분류에 적합하도록 다시 조합

[추가 합성곱층 #2] Conv2D(21, 1×1)
128개의 특징 맵을 조합하여 한국음식 21개 클래스에 대응하는 점수 맵 생성

[특징 맵 압축] GlobalAveragePooling2D
각 클래스에 대응하는 특징 맵의 공간적 평균을 계산하여 최종 21개의 로짓값(Logits)으로 변환
기존 CNN의 Flatten + Dense 방식 대신 Global Average Pooling을 이용하여 특징 맵을 직접 분류 점수로 변환

[출력층] Softmax - 21개 음식 다중분류
21개의 로짓값에 Softmax 활성화 함수를 적용하여 각 음식에 속할 예측 확률 계산
전체 확률의 합은 1(100%)이며, 가장 높은 확률의 음식을 최종 예측 결과로 선택 - argmax()

Tensorflow 02번 실습 - 전이학습을 이용한 한국음식 이미지 분류_폴더구조사용

[모델 구성] TensorFlow Keras Sequential API
Input(224,224,3) → EfficientNetV2B0 → Conv2D(128,1×1,ReLU) → Conv2D(21,1×1) → GlobalAveragePooling2D() → Softmax
사전학습된 CNN 모델을 하나의 레이어처럼 활용하고, 새로운 분류층을 추가하여 한국음식 전용 분류 모델 구성

[모델 구조 확장] Keras Functional API (요즘 권장)
Sequential API로 구현한 모델을 Functional API 구조로 변경하여 동일한 CNN 분류 모델 구현
Sequential API: 레이어를 순서대로 나열하여 모델 구성
Functional API: 각 레이어의 입력 Tensor와 출력 Tensor를 명시적으로 연결하여 모델 구성
tf.keras.Model(inputs=..., outputs=...)로 최종 모델 정의

[전이학습 및 반복 학습] SparseCategoricalCrossentropy + Optimizer
사전학습된 EfficientNetV2B0의 특징 추출기를 동결(Freeze)하고 새로 추가한 분류층부터 학습
필요하면 일부 사전학습 레이어의 동결을 해제하여 Fine-Tuning 수행

[새로운 이미지 예측] Google Drive 한국음식 사진 분류
학습에 사용하지 않은 새로운 한국음식 사진을 Google Drive에서 불러와 전처리 후 모델에 입력
model.predict()로 21개 음식의 예측 확률을 계산하고 가장 높은 확률의 음식 이름과 예측 확률 출력

이전 PyTorch 전이학습에서는 ResNet152의 마지막 FC 출력층을 교체하여 강아지 120종을 분류했다면, 이번 TensorFlow 실습에서는 EfficientNetV2B0의 특징 추출기를 재사용하고 Conv2D + GlobalAveragePooling2D 기반의 분류층을 추가하여 한국음식 21종 분류
또한 동일한 신경망을 Sequential API와 Functional API 두 방식으로 구성하여 TensorFlow Keras의 모델 설계 방법 비교
직접 수집한 이미지 데이터 준비 → 폴더 기반 자동 라벨링 → 사전학습 CNN 활용 → 새로운 분류층 학습 → Functional API 확장 → 실제 한국음식 이미지 예측까지 커스텀 CNN 전이학습의 전체 흐름 이해

Functional API 장점(권장하는 이유)

1. 복잡한 신경망 구조 구현 가능 (가장 큰 이유)
Sequential API는 '입력 1개 → 출력 1개'의 일방통행 구조만 가능. 반면 Functional API는 다음과 같은 복잡한 최신 논문의 구조를 모두 구현할 수 있음
다중 입력 (Multi-input): 예를 들어, 이미지 데이터와 텍스트 데이터를 동시에 입력받아 결과를 예측하는 모델을 만들 수 있음.
다중 출력 (Multi-output): 하나의 모델에서 물체의 '종류(분류)'와 '위치(회귀)'를 동시에 예측(예: Object Detection)할 수 있음.
잔차 연결 (Skip Connection / Residual Connection): ResNet 핵심 구조처럼 이전 층의 출력을 몇 개 층을 건너뛰고 뒤쪽 층의 입력에 더해주는 (Add/Concatenate) 구조를 쉽게 구성 가능

2. 레이어 공유 및 재사용의 편리성
동일한 가중치를 가진 레이어를 여러 번 재사용해야 할 때 강력
예) 두 개의 서로 다른 입력 이미지(A, B)를 동일한 특성 추출기(base_model)에 통과시켜 두 이미지의 유사도를 비교하는 '샴 네트워크(Siamese Network)' 같은 구조를 한 줄의 코드로 쉽게 연결

3. 모델 시각화 및 디버깅의 이점
Functional API로 짜여진 모델은 그래프 구조(Directed Acyclic Graph)로 내부가 정의됨
model.summary() 를 실행했을 때 각 레이어의 연결 상태(Connected to)가 명확히 표시되므로, 데이터가 어떤 경로로 흘러가는지 한눈에 파악하고 디버깅하기 좋음

4. 중간 레이어 추출 용이 (특성 맵 시각화 등)
모델 학습이 끝난 후, "이 모델의 3번째 층에서는 어떤 특징을 추출했을까?"가 궁금할 때 중간 출력층만 쏙 빼내어 새로운 모델로 정의하기가 용이함.
intermediate_model = tf.keras.Model(inputs=model.input, outputs=model.layers[3].output) 처럼 기존 모델의 부품을 그대로 가져와 활용할 수 있음.

사전학습 모델 선택

1. 사전학습 CNN 모델별 이미지 전처리 방식
사전학습 모델마다 요구하는 이미지 픽셀값 범위가 다르므로 모델에 맞는 전처리가 필요

입력범위 [0, 1] 사용하는 모델 : EfficientNet B0~B7, EfficientNetV2, MobileNetV3
[0, 1] 정규화 : 픽셀값 ÷ 255

입력범위 [-1, 1] 사용하는 모델 : ResNet50V2, MobileNetV2, InceptionV3 / Xception / NASNet
[-1, 1] 정규화 : 픽셀값 ÷ 127.5 - 1

전이학습에서는 학습과 추론에 동일한 이미지 전처리 방식을 적용해야 함

2. EfficientNet B0~B7 모델 비교
모델의 깊이, 너비, 입력 해상도를 함께 확장하여 정확도와 연산 효율을 개선한 CNN 모델

B0~B2: 가볍고 빠른 모델, 입문 및 제한된 GPU 환경에 적합
B3~B4: 정확도와 연산량의 균형
B5~B7: 높은 정확도, 많은 GPU 메모리와 연산량 필요

[이번 실습] EfficientNetV2B0
기존 EfficientNet을 개선한 별도 모델 계열로, 약 7.2M 파라미터와 224×224 입력 이미지 사용
데이터 규모, 분류 난이도, GPU 성능에 맞춰 적절한 사전학습 모델을 선택하는 것이 중요

모델	입력 크기	파라미터 수	ImageNet 정확도
B0	224×224	5.3M	77.1%
B1	240×240	7.9M	79.1%
B2	260×260	9.2M	80.1%
B3	300×300	12.3M	81.6%
B4	380×380	19.5M	82.9%
B5	456×456	30.6M	83.6%
B6	528×528	43.3M	84.0%
B7	600×600	66.7M	84.3%

Tensorflow 03번 실습 - 전이학습_한국음식 이미지분류 모델 불러오기

전이학습으로 만든 [한국음식 21종 분류 모델]을 저장하고 다시 불러와 [새로운 한국음식 사진]의 음식 종류를 예측하는 프로그램 만들기

이전 실습에서는 한국음식 이미지로 CNN 모델을 전이학습했다면, 이번에는 학습된 모델의 구조와 가중치를 파일로 저장하고 복원하여 추가 학습 없이 새로운 이미지의 음식 종류를 예측하는 추론(Inference) 과정 구현

[학습 모델 저장] model.save() + .keras
model.save("korean_food_model.keras")
TensorFlow Keras의 모델 저장 기능을 이용하여 학습된 CNN 모델을 파일로 저장
.keras: 모델 구조, 학습된 가중치, 컴파일 및 옵티마이저 상태 등을 저장하는 Keras 모델 파일
model.save(): 학습된 모델 전체를 저장
class_names: 21개 음식 클래스 번호와 이름의 매핑 정보를 별도로 저장

[모델 불러오기] tf.keras.models.load_model()
model = tf.keras.models.load_model("korean_food_model.keras")
저장된 모델의 구조와 가중치를 복원하여 학습된 EfficientNetV2B0 기반 한국음식 분류 모델 재사용
PyTorch의 load_state_dict() 방식과 달리 Keras는 저장된 모델의 구조와 가중치를 함께 복원할 수 있으므로 모델을 다시 설계할 필요가 없음

[새로운 이미지 입력] Google Drive 한국음식 사진
학습에 사용하지 않은 새로운 한국음식 사진(JPG, PNG 등)을 불러와 분류 수행
load_img(): 새로운 이미지 파일을 RGB 224×224 크기로 불러오기
img_to_array(): 이미지를 NumPy 배열로 변환
np.expand_dims(): 이미지 1장을 (1, 224, 224, 3) 형태의 배치 데이터로 변환
학습에 사용한 EfficientNetV2B0의 입력 전처리 방식과 동일하게 적용

Tensorflow 03번 실습 - 전이학습_한국음식 이미지분류 모델 불러오기

[이미지 특징 추출] EfficientNetV2B0
새로운 음식 사진을 사전학습된 CNN 특징 추출기에 통과시켜 색상, 재료, 질감, 음식 형태 등의 복합적인 시각적 특징 추출
학습된 가중치를 그대로 사용하여 역전파와 가중치 갱신 없이 순전파만 수행

[최종 분류] Softmax + argmax()
model.predict()로 21개 한국음식의 예측 확률 계산
Softmax: 21개 음식 클래스의 확률값 출력 (합계 100%)
np.argmax(): 가장 높은 확률을 가진 클래스 번호 선택
class_names[predicted_idx]: 클래스 번호를 실제 한국음식 이름으로 변환
예: 비빔밥 0.91 / 볶음밥 등 기타 음식 0.09 → 비빔밥(Bibimbap) 예측

[예측 결과 시각화] 이미지 + 음식명 + 예측 확률
입력한 한국음식 사진과 함께 예측 음식명, 예측 확률 출력
필요하면 상위 3개 음식의 예측 확률을 함께 출력하여 모델의 분류 결과 비교

이전 전이학습에서는 이미지 데이터 준비 → 모델 학습 → 가중치 최적화를 수행했다면,
이번 실습에서는 학습 모델 저장 → 모델 불러오기 → 새로운 이미지 전처리 → 순전파 → 음식 종류 예측이라는 추론 과정 구현
PyTorch에서는 주로 state_dict로 가중치를 저장하고 복원하지만, TensorFlow Keras에서는 model.save()와 load_model()로 모델 전체를 쉽게 저장하고 재사용
학습된 CNN 모델을 다시 학습하지 않고 새로운 사진 분류에 활용하여 실제 이미지 분류 서비스 개발의 기본 흐름 이해

(추가_선택) 이미지증강 전이학습

Tensorflow 03번 실습 - 전이학습_한국음식 이미지분류 모델 불러오기

수행과제

교재: 딥러닝 텐서플로 교과서

- 5장. 5.1~ 5.3 CNN, 전이학습 읽어보고 따라해보기 (p.142~ p.189)
- 5장. 5.4~ 5.5 읽어보기 (수업내용 외 내용. 참고용)
- 6장. 전이학습의 base model 이 된 대표적인 CNN 모델들 소개.
  AI 리서처들이 학습하는 내용. 이런 모델들이 논문으로 소개됨. 예전처럼 이런 모델을 완전히 학습할 필요는 없으며 대략적 특징들만 살펴보는 정도면 됨. 이미지분류를 위한 CNN, 객체인식을 위한 CNN, 이미지분할을 위한 CNN등이 소개되어 있음

개별과제: 이미지분류를 위한 무료 데이터셋을 찾아 본인만의 전이학습을 통한 이미지분류 모델 만들어보기

Ultralytics 데이터셋 : https://docs.ultralytics.com/ko/datasets#datasets-overview / https://platform.ultralytics.com/ultralytics?tab=datasets
AI Hub : https://www.aihub.or.kr/
Kaggle, Roboflow

[실무적용 사례 소개 Kakao Tech 블로그](https://tech.kakao.com/posts/623)